Class Assignment # 1

Student Name: Subhana Bushra

Registration Number: FA25-BDS-046

Class/Section: BDS-1A

Semester: 2nd

Department: Computer Science

Program: Data Science

Instructor: Sir Rizwan Rashid

Course: Object Oriented Programming

"E-commerce Management System"

1)- Explain how this system would be designed differently using the object-oriented approach compared to the modular/structural approach. Highlight the advantages of using classes and objects in this system.

Modular Approach:

- Data would be stored in separate data structures.
- Functions operate on these directly.
- Code is organized using functions.
- There is no direct relationship between data and functions that operate on them.

Object-Oriented Approach:

- Data and methods are encapsulated within classes(product, customer, order) which ensures security.
- Real world entities are modelled as objects with attributes and behaviours.
- Code reuse through inheritance.

Advantages of OOP in this system:

- Sensitive data can be protected using private access modifiers.
- Changes in one class don't affect others.
- Natural representation of business entities.

2)- Write a Product class with fields for productName, productID, and price. Implement methods to get and set the product details and explain how the state of a product object can change over time.

```java
public class Product {
    private String productName;
    private int productID;
    private double price;

    // Getter methids
    public String getProductName() {
        return productName;
    }

    public int getProductID() {
        return productID;
    }

    public double getPrice() {
        return price;
    }

    // Setter methods
    public void setProductName(String productName) {
        this.productName = productName;
    }

    public void setProductID(int productID) {
        this.productID = productID;
    }

    public void setPrice(double price) {
        if (price >= 0) {
            this.price = price;
        }
    }
}
```

A product's state can be changed over time using setter methods. Setter methods provide controlled access to modify private fields. Each setter call changes the object's internal state and without setters, object state would be frozen after creation. For example, product name might be changed in case of rebranding and price can be changed for sales and promotions.

3)- Create two constructors for the Product class: one default constructor and one parameterized constructor that initializes all fields. Demonstrate how to create Product objects using both constructors.

```
    // Default constructor
    public Product() {
        this.productName = "Unknown";
        this.productID = 0;
        this.price = 0.0;
    }

    // Parameterized constructor
    public Product(String productName, int productID, double price) {
        this.productName = productName;
        this.productID = productID;
        this.price = price;
    }

    // Creating objects using both constructors
    public static void main(String[] args) {

        Product product1 = new Product();
        product1.setProductName("Laptop");
        product1.setProductID(101);
        product1.setPrice(999.99);

        Product product2 = new Product("Smartphone", 102, 599.99);
    }
}
```

4)- Add another constructor that initializes only the productName and price fields. Explain how overloaded constructors improve flexibility in object creation.

```
  public Product(String productName, double price) {
    this.productName = productName;
    this.price = price;
}
```

Overloaded constructors provide flexibility in object creation by allowing multiple methods with the same name but different parameters, it promotes code reusability, simplifies the codebase, and improves code readability.

5)- Describe how memory is allocated for Product objects and how object references are managed when storing multiple Product objects in an array.

When you create a Product object in java, the actual object with all its fields is stored in heap memory whereas the variable holding it is stored in stack memory and contains a reference/ address of where the object lives in heap.

When we store multiple Product objects in an array:

```
Product[] catalog = new Produt[3];

catalog[0] = p1;
catalog [1] = p2;
catalog [2] = p3;
```

The array itself is stored in the stack but it doesn't hold the actual Product objects rather it holds the references to those objects in the heap. So catalog[0] is not the Laptop object itself, it's the pointer to where the Laptop object lives in heap memory. This means multiple variables can point to the same object. If you change actual object in heap, p1 will reflect the change too since they share same reference.

6)- For the Customer class, make all fields private and provide public getter and setter methods. Explain how encapsulation and visibility modifiers protect sensitive customer data (e.g., email).

```
class Customer{
    private String customerName;
    private int customerID;
    private String email;

    public String getEmail(){
        return email;
    }
    public void setEmail(String email){
        if( email.contains("@")){
            this.email = email;
        }
        else{
            System.out.println("Invalid Email!");
        }
    }
```

How encapsulation and visibility modifiers protect sensitive customer data?

1- Without private anyone can change emails of customers from anywhere in the code.
2- With private + setters, we have validation logic and we can control what values are allowed.
3- Getters give read access in a controlled way.

7)- Create a method in the Order class that takes a Product object as an argument to add it to an order. Use the this operator to differentiate between method parameters and class fields when adding the product.

```
class Order{
    private Product[] products;
    private int productCount;

    public void addProduct(Product product){
        //this.product is the array belonging to Order objet and product is
parameter passed into this method.
        this.product[this.productCount] = product;
        this.productCount++;
    }
}
```

Without 'this' keyword here, if both the filed and parameter had the same name, Java wouldn't know which one you're referring to. 'This' keyword overcomes shadowing.

8)- Implement a copy constructor in the Customer class to create a new customer object with the same details as an existing customer. Demonstrate how to use the copy constructor in the context of this system.

```
public Customer(Customer other){
    this.customerName = other.customerName;
    this.customerID = other.customerID;
    this.email = other.email;

    Customer c1 = new Customer("Ali khan" , 1, "ali@gmail.com" );
    Customer c2 = new Customer(c1);

    c2.setCustomerName("Sara khan.");
    System.out.println(c1.getCustomerName());  //still ali khan
    System.out.println(c2.getCustomerName()); // now sara khan

}
```

A copy constructor creates a new object by copying the data from an existing object of the same class. It is useful when you want a duplicated of a customer without them sharing the same memory addresss meaning changes to one won't affect the other.

This is different from just doing Customer c2 = c1. That would make c2 point to same object as c1 and changing c2 would also change c1. The copy constructor creates a completely independent object.

9)- Add static fields to the system to track the total number of customers and products. Implement static methods to display these counts and explain how static members are shared across all objects of the respective classes.

```java
class Product {
    private String productName;
    private int productID;
    private double price;

    // static field — ONE copy shared across ALL Product objects
    private static int totalProducts = 0;

    public Product(String productName, int productID, double price) {
        this.productName = productName;
        this.productID   = productID;
        this.price       = price;
        totalProducts++; // increments every time a new product is created
    }

    // static method — called on the class, not on any object
    public static int getTotalProducts() {
        return totalProducts;
    }
}

class Customer {
    private String customerName;
    private int customerID;
    private String email;

    private static int totalCustomers = 0;

    public Customer(String customerName, int customerID, String email) {
        this.customerName = customerName;
        this.customerID   = customerID;
        this.email        = email;
        totalCustomers++;
    }

    public static int getTotalCustomers() {
        return totalCustomers;
    }
```

```
public static void main(String[] args) {
    Product p1 = new Product("Laptop", 101, 999.99);   // totalProducts = 1
    Product p2 = new Product("Phone",  102, 499.99);   // totalProducts = 2
    Product p3 = new Product("Mouse",  103, 29.99);    // totalProducts = 3

    Customer c1 = new Customer("Ali Khan", 1, "ali@gmail.com");   //
totalCustomers = 1
    Customer c2 = new Customer("Sara Khan", 2, "sara@gmail.com"); //
totalCustomers = 2

    // called on the CLASS directly, not on any single object
    System.out.println("Total Products: "  + Product.getTotalProducts());   // 3
    System.out.println("Total Customers: " + Customer.getTotalCustomers()); // 2
}
```

A static field belongs to the class itself, not to any individual object. This means every object of that class shares the exact same copy of the static variable.

In this system, totalProducts is static because we want one single counter that increases every time any Product object is created. If it was a regular field, every Product would have its own separate counter, which makes no sense.

How static members are shared across all objects:

- When p1 is created → totalProducts becomes 1
- When p2 is created → totalProducts becomes 2
- When p3 is created → totalProducts becomes 3

All three objects p1, p2, p3 are looking at the same totalProducts variable in memory. None of them own it individually.

The static method getTotalProducts() is also called on the class (Product.getTotalProducts()) rather than on an object, because the count doesn't belong to any single product — it belongs to the class as a whole.

10)- Explain why customer names should be stored as immutable strings. Demonstrate how changing the customer's name results in the creation of a new string object rather than modifying the original object.

Why customer names should be stored as immutable Strings:

- *Data integrity*: Once a name is set, it can't be accidentally modified from some other part of the code. The original value is safe.
- *Security*: In a customer management system, you don't want a name value changing unexpectedly. Immutability prevents that.
- *Thread safety:* If multiple parts of the system read the name at the same time, immutability ensures they all see the same value.

```java
public static void main(String[] args) {

    // step 1 – create a string
    String name1 = "Ali Khan";
    String name2 = name1;  // name2 points to the SAME object as name1

    System.out.println("name1 = " + name1);  // Ali Khan
    System.out.println("name2 = " + name2);  // Ali Khan

    System.out.println("Same object? " + (name1 == name2));  // true
    // both are pointing to the exact same "Ali Khan" in memory

    // step 2 – now "update" name1
    name1 = "Ali Khan Updated";
    // Java did NOT change "Ali Khan"
    // it created a BRAND NEW string "Ali Khan Updated"
    // and made name1 point to it

    System.out.println("\nAfter update:");
    System.out.println("name1 = " + name1);  // Ali Khan Updated
    System.out.println("name2 = " + name2);  // Ali Khan is  still unchanged!

    System.out.println("Same object? " + (name1 == name2));  // false
    // now they point to different objects
}
```